CENG 201 – Task 1 Report

Topic: Patient List Implementation using Linked List

Time Complexity Analysis:

In this task, a singly linked list was implemented to manage admitted patients in a hospital system. The addPatient method adds a new patient to the beginning of the list.

Since we only need to update the head pointer and link the new node to the current head, the time complexity of this operation is O(1), where n is the number of patients in the list.

If we wanted to add to the end of the list, we would need to traverse from the head to the last node, which would result in O(n) complexity. However, by adding to the beginning, we achieve constant time complexity.

The removePatient method searches for a patient by ID and removes the corresponding node. In the worst case, the patient to be removed is located at the end of the list or does not exist, which requires traversal of the entire list. As a result, this operation has a linear time complexity of O(n).

Similarly, the findPatient method performs a linear search through the linked list to locate a patient by ID. Since each node may need to be checked, the time complexity of this operation is also O(n).

Linked List vs. ArrayList Comparison:

Compared to an ArrayList, a linked list offers more efficient insertions and deletions because it does not require shifting elements in memory. Removing a node from a linked list can be done by updating pointers, whereas removing an element from an ArrayList may require shifting multiple elements, resulting in O(n) time complexity.

However, accessing elements in an ArrayList by index has a time complexity of O(1), while accessing a specific element in a linked list requires traversal, making it O(n). In this hospital management scenario, where patients are continuously added and removed, a linked list is a suitable data structure choice.